

Agentic Research & Decision Intelligence Platform

Agentic Research & Decision Intelligence Platform

****Final Course Project Report****

****Topic:**** Multi-agent AI systems for trustworthy clinical decision support

****System:**** Agentic Research & Decision Intelligence Platform

****Mode:**** Deterministic mock/offline mode

****Run ID:**** `1bd06da9-bf3f-4903-a23b-a0a1ba025f62`

Abstract

This report presents a production-style prototype for an agentic multi-agent research and decision intelligence platform. The system accepts a complex user topic, plans the work, retrieves evidence, analyzes structured themes, critiques evidence quality, checks claims, generates figures, writes an academic-style report, evaluates quality with a rubric, and produces a presentation. The demonstration focuses on multi-agent AI systems for trustworthy clinical decision support, where hallucination risk, traceability, and human oversight are central design constraints. In deterministic mock mode, the platform uses a local corpus of verified references and produces reproducible outputs without paid API keys.

1. Introduction

Generative AI systems are increasingly used for research assistance, clinical summarization, policy analysis, and decision support. These tasks require more than a single conversational answer: they require planning, retrieval, structured analysis, critique, citation grounding, and transparent limitations. The platform built for this project treats research synthesis as a multi-agent workflow in which specialized agents exchange structured state rather than untracked free text.

2. Problem Statement

The target problem is: given a complex research or decision scenario, automatically produce a professional evidence-grounded report and presentation. The clinical decision support scenario is intentionally high stakes. The system must avoid implying autonomous diagnosis, must preserve source traceability, and must expose uncertainty. The prototype therefore asks each agent to produce inspectable intermediate artifacts and gives the critic and evaluator authority to trigger revisions.

3. Related Work

The design combines ideas from retrieval-augmented generation, tool-using language agents, critique/refinement loops, clinical decision support evaluation, and modern multi-agent orchestration frameworks. Retrieval grounding follows the RAG pattern introduced by Lewis et al. and related retrieval-memory work. Agentic control is inspired by ReAct, Reflexion, Self-Refine, and multi-agent systems such as AutoGen. Clinical safety requirements draw from clinical decision support reviews and reporting guidelines such as CONSORT-AI, SPIRIT-AI, DECIDE-AI, and WHO guidance on AI governance for health.

4. System Architecture

The system has five layers: user/API entry points, graph orchestration, specialized agents, tool adapters, and persistence/output storage. The FastAPI backend exposes health, run, and artifact endpoints. The Streamlit UI offers a demonstration surface for inspecting plans, agent notes, reports, figures, and evaluation scores. LangGraph is used when installed; a

deterministic fallback runner preserves the same conditional workflow for offline testing. This layered design separates orchestration policy from domain tools, making the prototype easier to test and extend.

****Figure 1. Overall System Architecture.**** See `report/assets/system\_architecture.md` for Mermaid source.

5. Multi-Agent Design

The platform implements ten agents: Planner, Research, Data Analyst, Critic, Fact-Checker, Report Writer, Visualization, Presentation, Memory, and Evaluator. Each agent receives and returns a `WorkflowState` object containing the query, plan, sources, tables, figures, checks, paths, scores, and agent notes. This makes the collaboration auditable and testable.

****Table 1. Agent Responsibilities****

Agent	Primary responsibility	Key output
---	---	---
Planner	Task decomposition and routing	Execution plan
Research	Evidence retrieval	Ranked sources
Analyst	Metrics, comparisons, formulas	Tables and insights
Critic	Weakness and risk review	Revision decision
Fact-checker	Claim grounding	Citation checks
Visualization	Figures and diagrams	PNG and Mermaid assets
Report writer	Academic synthesis	Final report
Evaluator	Rubric scoring	JSON/Markdown evaluation
Presentation	Stakeholder communication	Slides
Memory	Persistence	Run metadata

6. Agent Communication Protocol

Agents communicate through typed fields rather than hidden prompt history. Research returns `Source` objects; the analyst returns metrics and `AnalysisTable` objects; the critic returns `needs\_revision`; the fact-checker returns `ClaimCheck` objects; visualization returns `GeneratedFigure` objects. The protocol reduces implicit coupling and allows the API, UI, tests, and memory store to inspect the same state. Each agent appends an `AgentMessage` note, so the run record preserves both machine-readable artifacts and human-readable execution rationale.

****Figure 3. Agent Communication Protocol.**** See `report/assets/agent\_communication.md` for Mermaid source.

7. LangGraph Workflow Design

The workflow is intentionally not a simple linear pipeline. After analysis, the critic can route the state back to the Research Agent when evidence is insufficient. After report writing, the evaluator can route back to the Report Writer when the quality score is below threshold. This mirrors production review loops where outputs are revised before release.

****Figure 2. LangGraph Multi-Agent Workflow.**** See `report/assets/workflow.md` for Mermaid source.

8. Tool-Use Design

Tool functions are deliberately small: `search\_tools` loads mock or optional external search

results; ``citation_tools`` creates citation markers; ``visualization_tools`` produces Mermaid and chart artifacts; ``report_tools`` emits Markdown and PDF output; ``file_tools`` centralizes path-safe file writing. The tools are deterministic in mock mode, which keeps tests stable. When optional visualization or presentation packages are unavailable, the project falls back to valid SVG, PDF, and OpenXML artifacts rather than failing silently.

9. Memory and State Management

SQLite stores run metadata, paths, scores, and serialized state. Memory is not used to silently influence future reports in mock mode; instead, it provides traceability and lets ``/runs/{run_id}`` return saved run details. This avoids hidden cross-run contamination while still demonstrating persistent memory.

10. Retrieval and Citation Strategy

The mock corpus contains real, relevant references with clickable DOI or documentation URLs. The Research Agent ranks sources by query keyword overlap and credibility. The Fact-Checking Agent maps claims to source tags and flags unsupported claims. In a production deployment, external search APIs can be added behind the same ``search_sources`` interface.

11. Mathematical Formulation

Task decomposition is represented as:

```
```latex
```

$$T = \{t_1, t_2, \dots, t_n\}$$

```
```
```

where each subtask ``t_i`` is assigned to one or more agents with explicit dependencies.

The confidence score is:

```
```latex
```

$$C = \frac{\sum_{i=1}^n w_i s_i}{\sum_{i=1}^n w_i}$$

```
```
```

where ``s_i`` is an evidence or quality signal and ``w_i`` is its weight. In the demo, the computed confidence score is **95.5/100** based on source credibility, citation coverage, and evidence diversity.

The evaluation score is:

```
```latex
```

$$Q = \alpha F + \beta R + \gamma S + \delta C$$

```
```
```

where ``F`` is factuality, ``R`` is relevance, ``S`` is structure, and ``C`` is citation quality.

The implementation extends this with completeness, clarity, visual quality, and reproducibility.

12. Implementation Details

The repository is implemented in Python with typed modules, Pydantic-compatible schemas, FastAPI endpoints, Streamlit UI, SQLite persistence, Matplotlib-compatible visualizations, and Pytest tests. LangGraph is the preferred orchestrator because it exposes graph state and conditional edges clearly. The fallback runner mirrors the same behavior when LangGraph is unavailable, preserving reproducibility in constrained environments. The implementation avoids hardcoded secrets and keeps external-provider integration behind configuration and

tool boundaries.

13. Evaluation Methodology

The evaluator scores outputs from 0 to 100 across factuality, relevance, completeness, structure, citation quality, clarity, visual quality, and reproducibility. Scores are saved as JSON and Markdown. The evaluator is deliberately transparent: it does not claim clinical validation, only project-output quality.

****Table 5. Mock Evaluation Results****

| | | | |
|------------------|-------|---|--|
| Dimension | Score | Rationale | |
| --- | --- | --- | |
| Factuality | 86 | Claims are grounded in local source corpus | |
| Relevance | 90 | Plan and report stay on clinical decision support | |
| Completeness | 88 | Required sections, tables, figures, formulas included | |
| Citation quality | 84 | All key claims include citation markers | |
| Reproducibility | 95 | Mock mode, Docker, scripts, tests included | |

14. Results

The demo run collected ****21**** sources, generated ****14**** figures, wrote Markdown and PDF reports, produced a presentation, and saved evaluation files. The platform demonstrates agentic behavior through planning, conditional critique routing, fact checking, structured synthesis, and quality scoring. The output quality score is not a clinical validation result; it is a reproducibility and artifact-quality measure for this course project.

****Figure 4. Report Generation Pipeline.**** See `report/assets/report\_pipeline.md` for Mermaid source.

****Figure 6. Overall System Architecture.****

![[Overall System Architecture](assets/overall_system_architecture.png)]

****Figure 7. Conditional Multi-Agent Workflow.****

![[Conditional Multi-Agent Workflow](assets/multi_agent_workflow_diagram.png)]

15. Error Analysis

The main expected errors are stale external evidence, incomplete retrieval, weak claim-to-source matching, and overconfident wording in generated recommendations. The critic mitigates these errors by requiring bounded claims and explicit limitations. The fact-checker mitigates citation hallucination by using only source objects already present in state. For a real deployment, claim verification should move from tag matching to span-level entailment checks against source passages.

16. Limitations and Mitigation Strategies

****Table 4. Risk Controls****

| | | | |
|---------------------------|--|--|--|
| Risk | Control | Residual concern | |
| --- | --- | --- | |
| Hallucinated citation | Fact-checker requires source IDs | External URLs still need periodic verification | |
| Automation bias | Critic requires limitations and human review | Users may over-trust fluent reports | |
| Outdated medical evidence | Search tool supports web extension | Mock corpus is static | |

| Privacy leakage | Local SQLite and no secrets in code | Real deployments need PHI controls |
| Overfitting to rubric | Separate critique and evaluator agents | Rubric remains approximate |

17. Ethical Considerations

Clinical decision support systems can create automation bias, privacy risk, inequitable recommendations, and misplaced trust. This prototype is not a medical device and must not be used for patient-specific diagnosis. A production deployment would require institutional governance, privacy controls, clinical validation, monitoring, audit logs, and explicit human responsibility.

18. Future Work

Future work includes live web retrieval with source freshness scoring, vector search over institution-approved documents, stronger natural-language inference for claim verification, human-in-the-loop review checkpoints, role-based access control, and formal clinical evaluation. Additional agents could specialize in privacy review, regulatory mapping, and cost-benefit modeling.

19. Conclusion

The Agentic Research & Decision Intelligence Platform shows how a multi-agent architecture can convert a complex topic into an auditable research workflow. Its core contribution is the combination of structured state, explicit graph routing, critique and fact-checking loops, reproducible mock mode, professional report generation, and quality evaluation.

20. Framework Comparison

****Table 2. Framework Comparison****

| Framework | Strength | Trade-off | Best use |
| --- | --- | --- | --- |
| LangGraph | Typed graph orchestration with conditional edges | Requires explicit state design | Controlled multi-agent workflows |
| Google ADK | Code-first agent toolkit with Google ecosystem deployment | Younger ecosystem | Gemini/Vertex-oriented agents |
| AutoGen | Conversational multi-agent patterns | Can become chat-loop centric | Research prototypes and collaborative agents |
| CrewAI | Role/task abstraction is easy to teach | Less explicit low-level graph control | Sequential or crew-style automations |
| Semantic Kernel | Enterprise-friendly plugins and planners | SDK concepts can be broad | Microsoft ecosystem integration |

21. Claim Checks

- ****Supported:**** Retrieval grounding can reduce unsupported generation by forcing reports to cite external evidence. (Patrick Lewis, 2020); (Shahul Es, 2023). Supported by tagged source evidence.
- ****Supported:**** Clinical decision support requires human oversight and careful workflow integration. (Amit X. Garg, 2005); (Kensaku Kawamoto, 2005); (Reed T. Sutton, 2020). Supported by tagged source evidence.

- **Supported:** Conditional multi-agent graphs make critique and revision loops explicit. (Google, 2026); (Qingyun Wu, 2023); (Shunyu Yao, 2022). Supported by tagged source evidence.
- **Supported:** Evaluation should include factuality, relevance, structure, citation quality, and reproducibility. (Amit X. Garg, 2005); (Brennan Vasey, 2022); (Kensaku Kawamoto, 2005). Supported by tagged source evidence.
- **Supported:** The prototype is not a medical device and should not produce patient-specific diagnosis. (Brennan Vasey, 2022); (Reed T. Sutton, 2020); (World Health Organization, 2021). Supported by tagged source evidence.

22. Additional Tables

Table 3. Decision Criteria for Trustworthy Clinical AI

| Criterion | Operational metric | Target |
|-----------------|---|------------------------------|
| --- | --- | --- |
| Grounding | Share of claims with citations | >= 90% |
| Traceability | Report sections linked to sources and agent notes | Complete run record |
| Safety | Explicit uncertainty and escalation language | Required for clinical topics |
| Usability | Evidence summarized as tables and recommendations | Readable by non-engineers |
| Reproducibility | Mock mode, tests, Docker, saved outputs | One-command demo |

Table 4. Risk Controls

| Risk | Control | Residual concern |
|---------------------------|--|--|
| --- | --- | --- |
| Hallucinated citation | Fact-checker requires source IDs | External URLs still need periodic verification |
| Automation bias | Critic requires limitations and human review | Users may over-trust fluent reports |
| Outdated medical evidence | Search tool supports web extension | Mock corpus is static |
| Privacy leakage | Local SQLite and no secrets in code | Real deployments need PHI controls |
| Overfitting to rubric | Separate critique and evaluator agents | Rubric remains approximate |

Table 5. Mock Evaluation Results

| Dimension | Score | Rationale |
|------------------|-------|---|
| --- | --- | --- |
| Factuality | 86 | Claims are grounded in local source corpus |
| Relevance | 90 | Plan and report stay on clinical decision support |
| Completeness | 88 | Required sections, tables, figures, formulas included |
| Citation quality | 84 | All key claims include citation markers |
| Reproducibility | 95 | Mock mode, Docker, scripts, tests included |

23. Figures

- Figure 1. Overall System Architecture.** See `report/assets/system\_architecture.md` for Mermaid source.
- Figure 2. LangGraph Multi-Agent Workflow.** See `report/assets/workflow.md` for Mermaid

source.

****Figure 3. Agent Communication Protocol.**** See `report/assets/agent\_communication.md` for Mermaid source.

****Figure 4. Report Generation Pipeline.**** See `report/assets/report\_pipeline.md` for Mermaid source.

****Figure 5. Evaluation Pipeline.**** See `report/assets/evaluation\_pipeline.md` for Mermaid source.

****Figure 6. Overall System Architecture.****

![Overall System Architecture](assets/overall_system_architecture.png)

****Figure 7. Conditional Multi-Agent Workflow.****

![Conditional Multi-Agent Workflow](assets/multi_agent_workflow_diagram.png)

****Figure 8. LangGraph State Transition Diagram.****

![LangGraph State Transition Diagram](assets/langgraph_state_transition_diagram.png)

****Figure 9. Agent Communication Diagram.****

![Agent Communication Diagram](assets/agent_communication_diagram.png)

****Figure 10. Report Generation Pipeline.****

![Report Generation Pipeline](assets/report_generation_pipeline_diagram.png)

****Figure 11. Evaluation Pipeline.****

![Evaluation Pipeline](assets/evaluation_pipeline_diagram.png)

****Figure 12. Evidence Sources by Theme.****

![Evidence Sources by Theme](assets/evidence_by_theme.png)

****Figure 13. Prototype Quality Profile.****

![Prototype Quality Profile](assets/quality_radar.png)

****Figure 14. Conceptual Grounding Pipeline.****

![Conceptual Grounding Pipeline](assets/conceptual_grounding_pipeline.png)

24. Critic Notes

- Evidence coverage is adequate for a prototype, but deployment claims must remain bounded.
- The system should emphasize clinician oversight, auditability, and uncertainty rather than autonomous diagnosis.
- The mock corpus is suitable for reproducible testing but must be refreshed for real clinical use.

25. Reproducibility Notes

Run the project with `USE MOCK\_LLM=true` to reproduce deterministic outputs without paid API keys. The demo command is `python scripts/run\_demo.py`; tests are run with `pytest`; the API starts with `uvicorn app.main:app --reload`; the UI starts with `streamlit run app/ui/streamlit\_app.py`.

References

1. Reed T. Sutton, David Pincock, Daniel C. Baumgart, et al. (2020). *An Overview of Clinical Decision Support Systems: Benefits, Risks, and Strategies for Success*. <https://doi.org/10.1038/s41746-020-0221-y>
2. Amit X. Garg, Neill K. J. Adhikari, Heather McDonald, et al. (2005). *Effects of Computerized Clinical Decision Support Systems on Practitioner Performance and Patient

Outcomes*.

<https://doi.org/10.1001/jama.293.10.1223>

3. Kensaku Kawamoto, Caitlin A. Houlihan, E. Andrew Balas, David F. Lobach (2005).

Improving Clinical Practice Using Clinical Decision Support Systems: A Systematic Review of Trials.

<https://doi.org/10.1136/bmj.330.7494.765>

4. Brennan Vasey, Xiaoxuan Liu, et al. (2022). *DECIDE-AI: New Reporting Guidelines to Bridge the Development-to-Implementation Gap in Clinical Artificial Intelligence*.

<https://doi.org/10.1038/s41591-022-01772-9>

5. Google (2026). *Agent Development Kit Multi-Agent Systems Documentation*.

<https://google.github.io/adk-docs/agents/multi-agents/>

6. World Health Organization (2021). *Ethics and Governance of Artificial Intelligence for Health*. <https://www.who.int/publications/i/item/9789240029200>

7. Xiaoxuan Liu, Samantha Cruz Rivera, David Moher, et al. (2020). *CONSORT-AI Extension: Reporting Guidelines for Clinical Trials of Artificial Intelligence Interventions*.

<https://doi.org/10.1038/s41591-020-1034-x>

8. Samantha Cruz Rivera, Xiaoxuan Liu, An-Wen Chan, et al. (2020). *SPIRIT-AI Extension: Protocol Guidelines for Clinical Trials of Artificial Intelligence Interventions*. [[https://doi.org/10.1016/S2589-7500\(20\)30219-3](https://doi.org/10.1016/S2589-7500(20)30219-3)]([https://doi.org/10.1016/S2589-7500\(20\)30219-3](https://doi.org/10.1016/S2589-7500(20)30219-3))

9. Karan Singhal, Shekoofeh Azizi, Tao Tu, et al. (2023). *Large Language Models Encode Clinical Knowledge*.

<https://doi.org/10.1038/s41586-023-06291-2>

10. Ziwei Ji, Nayeon Lee, Rita Frieske, et al. (2023). *Survey of Hallucination in Natural Language Generation*. <https://doi.org/10.1145/3571730>

11. Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. (2023). *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*.

<https://doi.org/10.48550/arXiv.2308.08155>

12. Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*.

<https://doi.org/10.48550/arXiv.2005.11401>

13. Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*.

<https://doi.org/10.48550/arXiv.2210.03629>

14. Emily M. Bender, Timnit Gebru, Angelina McMillan-Major, Shmargaret Shmitchell (2021). *On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?*

<https://doi.org/10.1145/3442188.3445922>

15. Noah Shinn, Federico Cassano, Ashwin Gopinath, et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*.

<https://doi.org/10.48550/arXiv.2303.11366>

16. LangChain (2026). *LangGraph Graph API Documentation*.

<https://docs.langchain.com/oss/python/langgraph/graph-api>

17. Aman Madaan, Niket Tandon, Prakhar Gupta, et al. (2023). *Self-Refine: Iterative Refinement with Self-Feedback*. <https://doi.org/10.48550/arXiv.2303.17651>

18. Shahul Es, Jithin James, Luis Espinosa-Anke, Steven Schockaert (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. <https://doi.org/10.48550/arXiv.2309.15217>

19. Microsoft (2026). *Semantic Kernel Agent Framework Documentation*. <https://learn.microsoft.com/en-us/semantic-kernel/frameworks/agent/>

20. CrewAI (2026). *CrewAI Documentation*. <https://docs.crewai.com/>

21. Lilian Weng (2023). *LLM Powered Autonomous Agents*. <https://lilianweng.github.io/posts/2023-06-23-agent/>